Parallel Alpha-Beta Search for Chess: Root Splitting and a Hybrid Work Queue

Carter Griffin, Luke [Lname], Bryce Panza
School of Informatics, Computing & Cyber Systems
Northern Arizona University, Flagstaff, AZ, U.S.A.

Algorithm 1 Sequential negamax with alpha-beta pruning

```
1: function SEARCH(p, d, α, β)
2:     if d = 0 then return EVAL(p)
3:     for m ∈ MOVES(p) do
4:         p' ← MAKE(p, m)
5:         s ← −SEARCH(p', d − 1, −β, −α)
6:         if s ≥ β then return β
7:         α ← max(α, s)
8:     return α
```

***Abstract*—Alpha-beta pruning makes minimax game-tree search practical, but its pruning depends on bounds produced by earlier siblings, which forces a partial ordering on the search. This project intends to compare the effectiveness of several different parallel optimizations for the Alpha-beta pruning algorithm to determine their ability to overcome this sequential limitation. We will implement a sequential negamax alpha-beta search for chess in C and three pthreads-based parallel variants: static root splitting, dynamic root splitting through a shared FIFO work queue, and a hybrid that extends the work queue below the root in the manner of the Young Brothers Wait Concept. Each variant is measured against the sequential baseline for wall time, nodes searched, and speedup on up to 28 hardware threads.**

I. Project Idea

A chess engine selects a move by searching the tree of positions reachable from the current one and scoring the leaves with an evaluation function. Minimax search visits every node to a fixed depth. Alpha-beta pruning skips subtrees that cannot change the result, and with good move ordering it searches roughly the square root of the minimax node count [1]. Negamax is the same algorithm written with a single maximizing case, using $\max(a, b) = -\min(-a, -b)$ [1] (Algorithm 1).

A significant difficulty for parallelism is due a shared alpha-beta state across different tree nodes [2]. The window (α, β) passed to each child depends on the scores of the children searched before it. Alpha-beta skips moves that can not beat the best move found so far. Searching one at a time, each move benefits from knowledge of its predecessor, but searched in parallel, moves begin before predecessors yield that knowledge, so our total nodes pruned decreases (redundancy increases).

Algorithm 2 Dynamic root split (worker thread)

```
1: shared: queue Q of root moves, score α*, lock L
2: while POP(Q, m) do
3:     a ← α*
4:     p' ← MAKE(root, m)
5:     s ← −SEARCH(p', d − 1, −∞, −a)
6:     LOCK(L); α* ← max(α*, s); UNLOCK(L)
```

This project implements four parallel optimizations for the tree search on top of the same base environment:

1) **Sequential.** Algorithm 1, one thread. All speedups are reported relative to this version.
2) **Static root split.** The legal moves at the root are dealt round-robin to T threads before the search begins [2]. Each thread searches its moves with a private α. Results are combined after a join.
3) **Dynamic root split.** Root moves are placed in a shared FIFO queue. Idle threads take the next move. The best score found so far is shared, so moves taken later are searched with a tighter window (Algorithm 2).
4) **Hybrid.** The work queue of version 3 extended below the root. At a node, the first move is searched by the owning thread alone; once it returns a bound, the remaining moves are pushed to the shared queue as tasks. This follows the Young Brothers Wait Concept [3] and reuses the queue from version 3 instead of a per-node split-point structure (Algorithm 3).

II. Anticipated Challenges

The challenges follow from the dependence of each sibling's window on earlier siblings, and each version is affected differently.

Search overhead. Any node searched in parallel is searched with less information than in the sequential order. Static splitting has the most of this, since no thread sees another's α. The total node count at T threads, divided by the sequential node count, measures this cost directly.

Algorithm 3 Hybrid split at node n (sketch)

```
s_0 ← −SEARCH(child 0 of n)        ▷ eldest brother, sequential
update n.α; if cutoff then return
if n.depth ≥ D_split then
    push children 1..k of n to Q as tasks
    help process tasks from Q until all children of n finish
else
    search children 1..k sequentially
on any child result ≥ n.β: set n.abort
```

Load imbalance and idle time. Subtree sizes under different root moves vary by orders of magnitude. The first move in a well-ordered list should be the largest [1]. Under a static split, one thread can track most of the work while the others finish early and wait at the join. Comparing the different load balancing techniques available (dynamic, static, guided) will yield interesting results.

Limited parallelism at the root. A typical middlegame position has 30 to 40 legal moves [4]. Root splitting cannot use more threads than root moves, and in practice it is limited by the largest single subtree. This is the motivation for the hybrid.

Synchronization and contention. Each thread may write to the shared queue and shared α^*. In the hybrid, task pushes and pops occur at every split node, so the lock on Q is a likely bottleneck. Split depth D_{split} trades task granularity against contention. (add callback to state management discussed in introduction)

Cutoff propagation. In the hybrid algorithm, a beta cutoff at one child makes the remaining sibling tasks useless (they are already pruned!). Such tasks must be cancelled or abandoned through a flag that running threads can check. It is important to be cautious of lock overuse while approaching this problem, making it an interesting challenge for implementation.

Correctness and nondeterminism. As with any parallel program, parallel versions of our selected algorithms may visit nodes in different orders across runs. All versions must be tested rigorously for race conditions by running several thousand times each to ensure correctness.

III. Proposed Solution

The code is written in C with POSIX threads and built with gcc on Ubuntu. All four versions share the same board, move generation, evaluation, and move ordering. The only difference between them is the search parallelization approach; the outcome of which this project will measure. Positions are scored using a sum of standard piece values (pawn 1, knight 3, bishop 3, rook 5, queen 9) for each side, known as 'material count' [4].

Our comparison will evaluate the wall execution time for the four different algorithms across several different thread counts. This will be used to assess the parallel speedup provided by different algorithms and determine their efficiency as well as flat execution performance, such as to allude to redundancy and contention in the algorithms.

Static splitting uses one `pthread_create` per thread and a join. Dynamic splitting uses a mutex-protected array queue and a mutex-protected α^*; a variant reading α^* through a C11 atomic is a possible refinement. The hybrid adds a task structure (position, depth, window, parent pointer, abort flag) to the same queue, an atomic counter of outstanding children per split node, and a minimum split depth D_{split} chosen by experiment.

Experiments use a fixed set of test positions in FEN notation [5], searched to a fixed depth. For each version and $T \in \{1, 2, 4, 8, 16, 28\}$ the measurements are wall time, total nodes, speedup $S_T = t_{seq}/t_T$, efficiency S_T/T, and search overhead N_T/N_{seq}. Each configuration is run several times and the median is reported.

Table I
SEQUENTIAL BASELINE, 1 THREAD, DEPTH 8, BRANCHING FACTOR 35

Seed	Nodes	Time
1	1.68 B	5.16 s
2	1.82 B	5.60 s
3	1.97 B	6.05 s
4	1.84 B	5.65 s
5	1.47 B	4.54 s

IV. Preliminary Benchmark

We implemented a basic negamax algorithm to test sequential efficiency. The algorithm was run on a random tree model [1], [6] instead of from a potential game state. We plan to change the algorithm's implementation and use this as a general measurement for performance only. While not implemented against a legitimate chess board, the core negamax algorithm applies just as well to a tree of random leaves. We ran on one core of an Intel i7 14700K at depth 8 with seeds 1–5 (Table I), and got an average of 325M nodes per second evaluated. In each of our parallelizations, we hope to achieve a parallel speedup greater than 2, though we recognize the challenge of parallelizing alpha-beta implementations due to their inherent sequential nature [2]. Regardless, we intend to compare results between approaches to identify the most effective approach to parallelizing alpha-beta implementation.

V. Timeline

- Weeks 1–2: board, move generation, sequential search, test positions.
- Week 3: static and dynamic root splitting.
- Weeks 4–5: hybrid work queue, cutoff propagation, tuning D_{split}.
- Week 6: full measurements and final report.

References

[1] D. E. Knuth and R. W. Moore, "An analysis of alpha-beta pruning," *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.

[2] T. A. Marsland and M. Campbell, "Parallel search of strongly ordered game trees," *ACM Computing Surveys*, vol. 14, no. 4, pp. 533–551, 1982.

[3] R. Feldmann, B. Monien, P. Mysliwietz, and O. Vornberger, "Distributed game-tree search," *ICCA Journal*, vol. 12, no. 2, pp. 65–73, 1989.

[4] C. E. Shannon, "Programming a computer for playing chess," *Philosophical Magazine*, vol. 41, no. 314, pp. 256–275, 1950.

[5] S. J. Edwards, "Portable game notation specification and implementation guide," https://www.chessprogramming.org/Portable_Game_Notation, 1994.

[6] J. Pearl, "Asymptotic properties of minimax trees and game-searching procedures," *Artificial Intelligence*, vol. 14, no. 2, 1980.